

Green AI Benchmark Experimental Update

A controlled CPU comparison of equivalent Simple CNN training workloads in PyTorch and TensorFlow on CIFAR-10

PILOT RESULT

PyTorch completed the one-epoch run 2.01x faster and used 35.2% less tracked energy and estimated CO2e than TensorFlow in this specific environment.

29,194

TRAINABLE PARAMETERS
EACH

50,000

TRAINING IMAGES

782

BATCHES PER FRAMEWORK

CPU

CONTROLLED DEVICE

Research team

Samiul Ahmed Tamim

Md. Shakibul Hassan Prince

Tahmid Ahmed Talukder

Raisa Rahman

Department of Computer Science and Engineering
United International University
Dhaka, Bangladesh

Prepared 8 September 2026 | Benchmark run 7 September 2026 | CIFAR-10 training pilot

Executive Summary

This update reports the first completed controlled training pilot from the broader Green AI research plan. It compares one equivalent Simple CNN implementation in PyTorch and TensorFlow on the same CPU for one epoch.

2.01x

PYTORCH SPEEDUP

86.0006 s vs 42.8608 s

50.2%

SHORTER RUNTIME

relative to TensorFlow

35.2%

LOWER TRACKED ENERGY

0.706 Wh vs 1.089 Wh

35.2%

LOWER ESTIMATED CO2E

0.488 g vs 0.753 g

Metric	PyTorch	TensorFlow	Pilot finding
Trainable parameters	29,194	29,194	Exact count match
Execution time	42.8608 s	86.0006 s	PyTorch faster by 43.1398 s
Average training loss	1.5896	1.6005	Difference 0.0109
Tracked energy	0.705921 Wh	1.089164 Wh	PyTorch lower by 0.383243 Wh
Estimated emissions	0.488082 g CO2e	0.753060 g CO2e	PyTorch lower by 0.264978 g

The defensible conclusion

For this single CPU training run, the PyTorch implementation was faster and produced lower total tracked energy and lower estimated carbon emissions. The result is a useful pilot finding because the trainable parameter counts, dataset, batch size, optimizer type, learning rate, epoch count, normalization values, and machine were held constant.

The boundary of the conclusion

The run does not establish that PyTorch is universally greener. It includes one model, one epoch, one machine, and one run per framework. Test accuracy, inference, GPU behavior, repeated-run variance, peak power, and statistical significance were not measured. Average loss alone does not prove equivalent predictive performance.

Recommended presentation wording: In our first controlled Simple CNN CPU pilot, PyTorch was 2.01x faster and used 35.2% less tracked energy than TensorFlow. We treat this as a pilot result and will validate it with repeated, counterbalanced runs and accuracy measurements.

Research Context and Update Scope

The manuscript and Bangla project report define a wider study. This update marks the first implemented full-model training experiment within that program.

Research objective

The project asks when equivalent deep learning workloads in PyTorch or TensorFlow deliver lower energy use and estimated carbon impact while maintaining comparable predictive quality. The planned study separates framework effects from differences in architecture, data, training configuration, hardware, and measurement method. [S1, S2]

Study element	Planned research program	Completed in this update
Models	Simple CNN, ResNet-18, MobileNetV2, EfficientNet-B0, Vision Transformer	Simple CNN
Datasets	MNIST for pilots; CIFAR-10 for image classification	CIFAR-10 training split
Workload phases	Training and inference	Training only
Devices	Controlled CPU and GPU configurations	CPU only
Measurement	Energy, power, CO2e, runtime, utilization, memory, throughput, loss, accuracy	Runtime, loss, CodeCarbon energy and estimated CO2e; some utilization metadata
Repetition	At least five runs per condition with statistics	One run per framework
Analysis levels	Full model plus operator and mini-pipeline benchmarks	Full-model end-to-end training

Why this pilot matters

- It verifies that both frameworks can implement the same trainable topology with an exact parameter-count match.
- It confirms that CodeCarbon can collect CPU and RAM energy data through the Windows Energy Meter Interface on the target machine.
- It produces a first directional comparison and exposes protocol issues that should be fixed before the repeated study.
- It creates a reproducible reporting path from console output and CSV measurements to a presentable research update.

Status: protocol concept completed; pilot instrumentation operational; first Simple CNN CPU pair completed; confirmatory study still pending.

Experimental Configuration

Both frameworks processed the same CIFAR-10 training workload on the same CPU. The run was designed as an end-to-end framework comparison, including each framework's data pipeline and training loop.

Category	Configuration
Hardware	AMD Ryzen 5 2600 Six-Core Processor; 12 logical threads; 15.927 GB RAM; no GPU detected
Operating environment	Windows 10 build string 10.0.26100; Python 3.11.4
Frameworks	PyTorch 2.14.0+cpu; TensorFlow 2.21.0
Measurement	CodeCarbon 3.3.0; OfflineEmissionsTracker; country code BGD; CPU via Windows EMI; RAM via 10 W estimation model
Dataset	CIFAR-10 training split; 50,000 images; 10 classes; input size 32 x 32 x 3
Training	1 epoch; batch size 64; 782 batches; Adam; learning rate 0.001; random seed 42
Preprocessing	Scale to [0,1], then channel normalization with mean (0.4914, 0.4822, 0.4465) and standard deviation (0.2023, 0.1994, 0.2010)

Controls that were implemented

Implemented control	Evidence
CPU-only execution for both frameworks	Script and terminal output [S3, S5]
Identical model layer sequence and trainable parameter count	Script and terminal output [S3, S5]
Same dataset size, normalization constants, batch size, epoch count, optimizer family, and learning rate	Script and terminal output [S3, S5]
Fixed framework-specific random seeds and reproducible shuffle settings within each pipeline	Script and terminal output [S3, S5]
Dataset download and setup completed before the timed training regions	Script and terminal output [S3, S5]

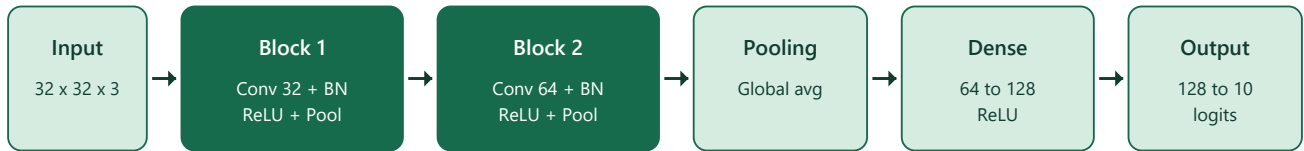
Controls that remain incomplete

The code does not force identical initial weights or an identical cross-framework sample order. Batch normalization and Adam rely on framework defaults that are not numerically identical. CPU thread counts, warm-up policy, cooling period, background load, and run order were also not controlled explicitly.

Model Architecture and Parameter Parity

The Simple CNN was matched layer by layer and both implementations reported 29,194 trainable parameters.

Equivalent trainable topology in both frameworks



Batch normalization moving statistics were excluded from the TensorFlow trainable count.

Stage	Parameter calculation	Trainable parameters
Conv 3 to 32 plus BatchNorm	$3 \times 32 \times 3 \times 3 + 32$ bias; 32 scale + 32 offset	960
Conv 32 to 64 plus BatchNorm	$32 \times 64 \times 3 \times 3 + 64$ bias; 64 scale + 64 offset	18,624
Dense 64 to 128	$64 \times 128 + 128$ bias	8,320
Dense 128 to 10	$128 \times 10 + 10$ bias	1,290
Total	Exact match in both frameworks	29,194

Parity strengths

- Convolution sizes, channel counts, activations, pooling, global average pooling, dense widths, logits, and trainable parameter totals match.
- TensorFlow trainable parameters were counted directly so non-trainable BatchNormalization moving statistics did not inflate the comparison.

Parity caveats

- PyTorch BatchNorm defaults use momentum 0.1 and epsilon 1e-5; TensorFlow defaults use momentum 0.99 and epsilon 1e-3. These settings change training behavior even when the trainable counts match.
- Setting the same numeric seed in two frameworks does not generate identical initial tensors. A strict paired benchmark should export one initialization and load equivalent weights into both models.
- PyTorch and TensorFlow use different shuffle algorithms, so equal seeds do not guarantee the same image order across frameworks.

Parameter parity is necessary, but it is not sufficient for full computational equivalence. The current run demonstrates matched trainable size and near-matched high-level architecture.

Benchmark Execution Timeline

The console log shows the benchmark progressing from environment checks to two sequential tracked training runs. Dataset download occurred before timing.

Time or stage	Observed terminal event	Meaning
Environment check	PyTorch 2.14.0+cpu; TensorFlow 2.21.0; ALL PACKAGES OK	Required packages imported successfully.
Compile check	python -m py_compile completed without output	The benchmark script passed Python syntax compilation.
Dataset setup	50,000 samples in each pipeline; TensorFlow archive download took 1767 s	The long download was outside the timed training regions and is not included in runtime results.
22:11:19 to 22:12:06	PyTorch tracker initialized; batches 100 through 700 reported; run completed	782 batches trained in 42.8608 s by the script timer.
22:12:06 to 22:13:33	TensorFlow tracker initialized; batches 100 through 700 reported; run completed	782 batches trained in 86.0006 s by the script timer.
Final output	Parameter, time, loss and CO2e table printed; Word report saved	Automated reporting finished successfully.

Terminal messages explained

- The oneDNN message is informational. TensorFlow reports that optimized CPU kernels may produce small floating-point differences because operation order can change.
- No GPU found confirms that both runs were CPU-only.
- CPU Tracking Method Windows EMI means CodeCarbon used the Windows energy meter interface for CPU energy.
- RAM Power 10.0 W is a model-based estimate, not a directly measured DRAM power sensor.
- The multiple instances warning is a general CodeCarbon notice; the transcript shows sequential PyTorch and TensorFlow runs, not overlapping training loops.

Timing boundary: tracker.start() runs before time.perf_counter(), and tracker.stop() runs after the ending timer. This explains the small difference between script time and CodeCarbon duration.

Measured Results

The raw measurements show a consistent total-runtime and total-energy advantage for PyTorch in this run.

Training runtime

PyTorch



TensorFlow



Lower is better

Tracked energy

PyTorch



TensorFlow



Lower is better

Metric	PyTorch	TensorFlow	Difference
Script execution time	42.8608 s	86.0006 s	43.1398 s lower for PyTorch
CodeCarbon duration	42.9578 s	86.0100 s	43.0522 s lower for PyTorch
Average training loss	1.5896	1.6005	0.0109 lower for PyTorch
CPU energy	0.586891 Wh	0.850344 Wh	0.263453 Wh lower for PyTorch
RAM energy	0.119030 Wh	0.238820 Wh	0.119790 Wh lower for PyTorch
Total tracked energy	0.705921 Wh	1.089164 Wh	0.383243 Wh lower for PyTorch
Estimated emissions	0.488082 g CO ₂ e	0.753060 g CO ₂ e	0.264978 g lower for PyTorch

Direct observations

- PyTorch completed the epoch in 42.8608 seconds; TensorFlow required 86.0006 seconds.
- CodeCarbon recorded 0.705921 Wh for PyTorch and 1.089164 Wh for TensorFlow.
- Estimated emissions were 0.488082 g CO₂e and 0.753060 g CO₂e respectively.
- The carbon difference mirrors the energy difference because both runs used the same offline Bangladesh location setting and carbon-intensity basis.

Efficiency Calculations and Interpretation

Derived metrics make the mechanism visible: PyTorch used more tracked power on average but finished much sooner, so its total energy and emissions were lower.

Derived metric	PyTorch	TensorFlow	Interpretation
Relative speed	2.0065x vs TF	1.0000x	PyTorch processed the epoch about twice as fast.
Approx. throughput	1166.6 images/s	581.4 images/s	Derived from 50,000 images divided by script runtime.
Energy per image	0.05083 J	0.07842 J	PyTorch used 35.2% less tracked energy per training image.
Derived avg. tracked power	59.16 W	45.59 W	Total CPU+RAM tracked energy divided by tracker duration.
Emissions rate	11.362 mg/s	8.755 mg/s	PyTorch's rate was higher, but its runtime was much shorter.
Loss gap	-0.0109	reference	Small in this run; not a substitute for test accuracy.

DERIVED METRIC FORMULAS

Runtime speedup = 86.0006 / 42.8608 = 2.0065x
Runtime reduction = (86.0006 - 42.8608) / 86.0006 = 50.16%
Energy reduction = (1.089164 - 0.705921) / 1.089164 = 35.19%
Emissions reduction = (0.753060 - 0.488082) / 0.753060 = 35.19%
PyTorch energy per image = 0.705921 Wh x 3600 / 50,000 = 0.05083 J

Why total energy fell

The PyTorch run averaged about 59.16 W across the tracked CPU and RAM energy, compared with about 45.59 W for TensorFlow. PyTorch therefore operated at a higher average tracked power level, but it completed in roughly half the time. The shorter duration more than offset the higher power, producing lower total energy. This is a race-to-idle pattern within this single run.

Energy and emissions are totals. A lower instantaneous power draw does not guarantee a lower-energy job when the job runs much longer.

Selected Terminal Evidence

These excerpts preserve the decisive checkpoints from the supplied console output. Explanations appear below each excerpt.

1 ENVIRONMENT AND PARAMETER CHECK

```
PyTorch: 2.14.0+cpu
TensorFlow: 2.21.0
ALL PACKAGES OK

PyTorch trainable parameters : 29,194
TensorFlow trainable parameters: 29,194
Parameter parity: MATCH
```

The framework imports succeeded and both models reported the same number of trainable parameters.

2 DATASET CHECK

```
LOADING CIFAR-10 DATASET
PyTorch training samples: 50,000
TensorFlow training samples: 50,000
Dataset preparation complete.
```

Both pipelines contained the complete CIFAR-10 training split. The archive download occurred before the training timers started.

3 PYTORCH RUN

```
RUNNING PYTORCH BENCHMARK - 1 EPOCH
No GPU found.
CPU Tracking Method: Windows EMI
PyTorch batch 700/782 | Loss: 1.4064

Execution time : 42.8608 seconds
Average loss : 1.5896
CO2 emissions : 0.00048808 kg CO2eq
```

The tracker used CPU and estimated RAM energy; the loop completed all 782 batches.

Selected Terminal Evidence Continued

The TensorFlow completion and final comparison confirm the values used in the analysis. The environment block records the measurement context.

4 TENSORFLOW RUN AND FINAL COMPARISON

```
RUNNING TENSORFLOW BENCHMARK - 1 EPOCH
TensorFlow batch 700/782 | Loss: 1.3706

Execution time : 86.0006 seconds
Average loss   : 1.6005
CO2 emissions  : 0.00075306 kg CO2eq

Faster framework: PyTorch
Execution-time difference: 43.1398 seconds
Lower estimated emissions: PyTorch
```

The final console table agrees with the generated Word report and with the leading numerical fields in the CSV.

5 CODECARBON ENVIRONMENT METADATA

```
Platform system : Windows-10-10.0.26100-SP0
Python version  : 3.11.4
CodeCarbon      : 3.3.0
Available RAM   : 15.927 GB
CPU count       : 12 thread(s) in 1 physical CPU
CPU model      : AMD Ryzen 5 2600 Six-Core Processor
GPU count       : 0
Country code    : BGD
```

This metadata was printed for both tracked runs. Because the TensorFlow CSV row is malformed after the location fields, the console is the reliable source for these details.

Implementation Evidence for Fairness

Representative excerpts show how the benchmark fixed the device, data configuration, and parameter-count comparison. The snippets are cleaned to valid Python notation from the executed script.

CONFIGURATION AND DEVICE

```
SEED = 42
BATCH_SIZE = 64
NUM_CLASSES = 10
LEARNING_RATE = 0.001

torch.manual_seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

# Both frameworks intentionally run on CPU.
device = torch.device("cpu")
```

PARAMETER PARITY CHECK

```
# Count only trainable parameters.
pt_params = sum(
    p.numel() for p in pt_model_temp.parameters()
    if p.requires_grad
)

tf_params = sum(
    int(np.prod(variable.shape))
    for variable in tf_model_temp.trainable_variables
)

parity_status = "Match" if pt_params == tf_params else "Mismatch"
```

MATCHED NORMALIZATION

```
transform_pt = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        mean=(0.4914, 0.4822, 0.4465),
        std=(0.2023, 0.1994, 0.2010),
    ),
])

def preprocess_tf(image, label):
    image = tf.cast(image, tf.float32) / 255.0
    image = (image - tf_mean) / tf_std
    return image, label
```

What these excerpts establish

The code fixes the high-level experimental inputs and compares trainable parameter counts correctly. It does not yet create cross-framework identity for weight initialization or shuffled sample order, and it does not set every framework-specific default explicitly.

Implementation Evidence for Measurement

The measured region begins after tracker initialization and ends before tracker shutdown. Each framework uses a manual training loop for one epoch.

PYTORCH MEASUREMENT REGION

```
tracker_pt = OfflineEmissionsTracker(
    country_iso_code="BGD",
    project_name="PyTorch_SimpleCNN",
)
tracker_pt.start()
start_pt = time.perf_counter()

for inputs, targets in pt_loader:
    optimizer.zero_grad()
    outputs = pt_model(inputs)
    loss = criterion(outputs, targets)
    loss.backward()
    optimizer.step()

end_pt = time.perf_counter()
emissions_pt = tracker_pt.stop()
```

TENSORFLOW MEASUREMENT REGION

```
tracker_tf = OfflineEmissionsTracker(
    country_iso_code="BGD",
    project_name="TensorFlow_SimpleCNN",
)
tracker_tf.start()
start_tf = time.perf_counter()

for x_batch, y_batch in tf_dataset:
    with tf.GradientTape() as tape:
        logits = tf_model(x_batch, training=True)
        loss_value = tf_loss_fn(y_batch, logits)
        grads = tape.gradient(loss_value, tf_model.trainable_weights)
        tf_optimizer.apply_gradients(zip(grads, tf_model.trainable_weights))

end_tf = time.perf_counter()
emissions_tf = tracker_tf.stop()
```

AUTOMATED COMPARISON

```
if pt_duration < tf_duration:
    faster_framework = "PyTorch"
    speed_difference = tf_duration - pt_duration
else:
    faster_framework = "TensorFlow"
    speed_difference = pt_duration - tf_duration

greener_framework = (
    "PyTorch" if emissions_pt < emissions_tf else "TensorFlow"
)
```

Measurement interpretation

This timing includes forward pass, loss calculation, backpropagation, optimizer updates, and the framework-specific input pipeline during iteration. Tracker setup and shutdown are outside the script timer, so the CodeCarbon durations are slightly longer.

Research Interpretation

The pilot answers a narrow version of the project question and establishes the next methodological priorities.

Evidence	Supported interpretation	Claim not yet supported
29,194 trainable parameters in both models	The implementations have equal trainable size and matched high-level topology.	The numerical computations are identical at every step.
42.8608 s vs 86.0006 s	PyTorch was about 2.01x faster for this CPU training epoch.	PyTorch is always faster on other CPUs, GPUs, models, or versions.
0.705921 Wh vs 1.089164 Wh	PyTorch used 35.2% less tracked total energy in this run.	PyTorch always uses less energy.
0.488082 g vs 0.753060 g CO2e	PyTorch had lower CodeCarbon-estimated emissions under the same BGD setting.	These values are direct physical emissions measurements.
Loss 1.5896 vs 1.6005	Training losses were close after one epoch.	The models achieved equal test accuracy or convergence.

Answer for the current pilot

Under the tested conditions - Simple CNN, CIFAR-10, batch size 64, Adam at 0.001, one epoch, CPU-only execution on the AMD Ryzen 5 2600 - PyTorch delivered the better observed runtime and total tracked energy result. TensorFlow drew less average tracked power but ran long enough to consume more total energy.

Connection to the broader paper

The result supports the paper's central premise that framework execution can produce different efficiency outcomes even when high-level workloads are closely matched. It does not resolve the broader research question because the proposal explicitly calls for multiple architectures, training and inference, repeated measurements, operator-level analysis, predictive performance, and statistical testing. [S1, S2]

The most credible scientific message is conditional: PyTorch led this first CPU pilot; the project now needs repeated and stricter paired experiments to determine whether the effect is stable.

Validity Review and Data Quality

The pilot has a strong controlled core, but several issues could affect the size or repeatability of the observed difference.

Issue	Why it matters	Priority
One run per framework	No variance, confidence interval, or significance test can be computed.	High
Fixed run order	PyTorch always ran first; thermal state, caching, or background activity may favor one framework.	High
No warm-up policy	TensorFlow tracing or initialization and cold caches can influence the first measured epoch.	High
No test accuracy	Average training loss cannot confirm comparable predictive performance.	High
Different framework defaults	BatchNorm momentum/epsilon and Adam epsilon are not explicitly aligned.	High
Non-identical initialization/order	Equal seeds do not produce identical cross-framework weights or batch order.	Medium
CPU only	The result does not describe CUDA, GPU kernels, or deployment inference.	Medium
Estimated carbon	CodeCarbon converts tracked energy through an offline location factor; emissions are not directly measured.	Medium
CSV row shape	The TensorFlow row has 35 values under a 38-column header, shifting later metadata fields.	Medium

CSV integrity finding

The header and PyTorch row each contain 38 fields, while the TensorFlow row contains 35. The TensorFlow runtime, emissions, energy, country, and country code appear before the malformed region and remain usable. Hardware and software details for TensorFlow should be taken from the terminal metadata in this update, not from the shifted tail of that CSV row.

This issue does not change the reported TensorFlow runtime, total energy, or emissions, but it should be corrected before automated multi-run analysis.

Recommended Confirmatory Experiment

The next run should preserve the useful parts of the pilot and remove the largest sources of bias.

Step	Protocol improvement	Expected benefit
1	Run one untracked warm-up epoch or fixed warm-up batches for each framework.	Reduces startup, tracing, and cold-cache effects.
2	Run at least five paired repetitions and alternate or randomize framework order.	Enables variance estimates and reduces order bias.
3	Set CPU thread counts, power plan, background-process policy, idle period, and temperature logging.	Improves machine-state consistency.
4	Export a shared training index order and map one initialization into both frameworks.	Creates a stricter paired workload.
5	Set BatchNorm momentum/epsilon and Adam beta/epsilon values explicitly.	Removes hidden framework-default differences.
6	Record training and test accuracy, final loss, throughput, peak RAM, and average/peak power.	Supports a complete energy-performance comparison.
7	Write separate CodeCarbon CSV files per framework/run and validate column counts immediately.	Prevents mixed-schema output and simplifies audit.
8	Report mean, median, standard deviation, 95% confidence interval, paired effect size, and raw points.	Shows whether the observed difference is stable and meaningful.
9	Repeat for inference and the planned ResNet-18, MobileNetV2, EfficientNet-B0, and ViT workloads.	Connects the pilot to the full research claim.

Decision Rule and Next Deliverables

The repeated study should define success before collecting the full result set and preserve every run in a validated structure.

Minimum decision rule

Declare a configuration greener only when predictive performance is within a pre-specified tolerance and the repeated energy-per-sample result is lower with uncertainty reported. The project proposal suggests an example accuracy tolerance of one percentage point, but the team should confirm the threshold before collecting the full dataset. [S2]

Suggested next deliverables

- A validated long-format results file with one row per run and a schema check.
- Runtime, energy, emissions, throughput, loss, and accuracy plots with raw run points.
- A configuration file containing every controlled value and library version.
- A one-page comparison for each model plus an aggregate framework summary.

Reproducibility Record

The identifiers and raw measurements below allow this pilot to be traced back to the supplied files.

Field	Recorded value
Experiment ID	5b0fa12a-3dd7-45bb-9766-cc326314d9f1
PyTorch run ID	50c3a499-364a-4b29-8653-550629a01452
TensorFlow run ID	15c50b4f-4ef1-4058-bb1b-882aa2a69b5a
CSV end timestamps	PyTorch 2026-09-07 22:12:06; TensorFlow 2026-09-07 22:13:33
Project names	PyTorch_SimpleCNN; TensorFlow_SimpleCNN
Carbon location	Bangladesh; ISO code BGD
Power usage effectiveness	1.0 in the PyTorch row; TensorFlow tail fields malformed
Water usage effectiveness	0.0 in the PyTorch row; no water consumed was reported

Source register

Source	Role in this update
S1 Green_Computing_draft.pdf	English paper manuscript: official title, authors, research gap, planned scope, and literature foundation.
S2 Green_AI_Benchmark_Project_Report_Bangla.pdf	Detailed Bangla project protocol: research questions, controls, measurement plan, limitations, statistics, and work plan.
S3 Green_AI_Benchmark_Report.docx	Automatically generated one-run benchmark summary and raw comparison table.
S4 emissions.csv	CodeCarbon energy, emissions, duration, run identifiers, hardware metadata, and CSV schema issue.
S5 Supplied terminal transcript and benchmark code	Framework versions, execution sequence, batch progress, timing, loss, emissions, warnings, and implementation logic.

Audit note

The script runtime and CodeCarbon duration differ by 0.0970 s for PyTorch and 0.0094 s for TensorFlow because their timing boundaries are not identical. The Word report correctly uses the script timers and the emission values returned by `tracker.stop()`.

Presentation Summary

A concise sequence for explaining the update to a supervisor.

Talking point	Suggested explanation
1 Research question	We are testing whether equivalent deep learning workloads use different amounts of time and energy in PyTorch and TensorFlow.
2 Fairness setup	For this pilot we used the same Simple CNN trainable size, CIFAR-10 data, batch size, optimizer family, learning rate, epoch count, normalization, CPU, and random seed value.
3 Main result	PyTorch trained the epoch in 42.8608 seconds versus 86.0006 seconds and used 0.705921 Wh versus 1.089164 Wh.
4 Carbon result	With the same CodeCarbon Bangladesh setting, PyTorch was estimated at 0.488082 g CO2e and TensorFlow at 0.753060 g CO2e.
5 Key insight	PyTorch drew more tracked power on average but finished fast enough to reduce total energy by 35.2%.
6 Scientific caution	This is one CPU run, not a universal ranking. We have not yet measured test accuracy, inference, GPU behavior, repeated-run variance, or statistical significance.
7 Next step	We will repeat counterbalanced runs with warm-up, stricter cross-framework parity, accuracy evaluation, validated CSV output, and additional model architectures.

One-sentence conclusion

In the first controlled Simple CNN CPU pilot, PyTorch delivered roughly twice the training throughput and 35.2% lower tracked energy and estimated CO2e than TensorFlow, while the next study phase will test whether that advantage persists under repeated and stricter paired conditions.

End of experimental update